

MEMORY MANAGEMENT

One of the most-demanded resources in computers (other than the CPU) is system memory. Every process needs it since a process' code, stack, heap (dynamically-allocated structures), and data (variables) must all reside in memory. A processor reads instructions from memory and reads/writes data from/to memory. The functional interface of memory is:

- *value* = read(*address*): read the contents of memory location at *address*
- write(*address*, *value*): write *value* to the memory location at *address*

The memory manager is the part of the operating system that is responsible for allocating this resource to processes.

Memory is a storage area on the computer that contains the instructions and data that the computer uses to run the applications.

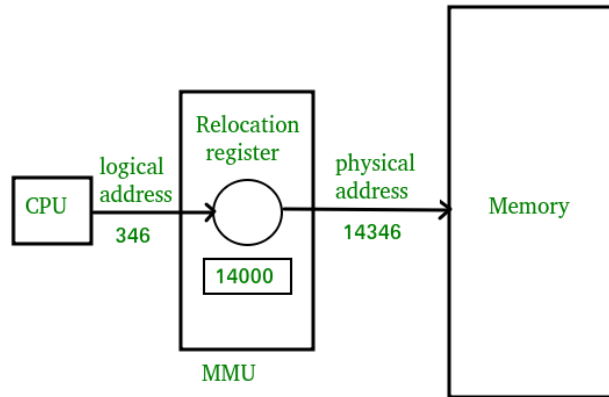
When the applications or the [operating system](#) need more memory than is available on the computer, the system must swap the current contents of the memory space with the contents of the memory space that is being requested. In the same way, different situations need different memory management techniques.

LOGICAL VERSUS PHYSICAL ADDRESSES IN AN OPERATING SYSTEM

A logical address is generated by CPU while a program is running. Since a logical address does not physically exist it is also known as a virtual address. This address is used as a reference by the CPU to access the actual physical memory location.

There is a hardware device called Memory-Management Unit is used for mapping logical address to its corresponding physical address.

A physical address identifies the physical location of a specific data element in memory. The user never directly deals with the physical address but can determine the physical address by its corresponding logical address. The user program generates the logical address and believes that the program is running in this logical address space, but the program needs physical memory for its execution, therefore, the logical address must be mapped to the physical address by the MMU before the addresses are used. The term physical address space is used for all physical addresses corresponding to the logical addresses in a logical address space.



Differences between Logical and Physical Address in Operating System

1. The basic difference between Logical and physical address is that Logical address is generated by CPU in perspective of a program whereas the physical address is a location that exists in the memory unit.
2. Logical Address Space is the set of all logical addresses generated by CPU for a program whereas the set of all physical address mapped to corresponding logical addresses is called Physical Address Space.
3. The logical address does not exist physically in the memory whereas physical address is a location in the memory that can be accessed physically.
4. Identical logical addresses are generated by Compile-time and Load time address binding methods whereas they differs from each other in run-time address binding method.
5. The logical address is generated by the CPU while the program is running whereas the physical address is computed by the Memory Management Unit (MMU).

Comparison Chart:

Parameter	LOGICAL ADDRESS	PHYSICAL ADDRESS
Basic	generated by CPU	location in a memory unit
Address Space	Logical Address Space is set of all logical addresses generated by CPU in reference to a program.	Physical Address is set of all physical addresses mapped to the corresponding logical addresses.
Visibility	User can view the logical address of a program.	User can never view physical address of program.
Generation	generated by the CPU	Computed by MMU
Access	The user can use the logical address to access the physical address.	The user can indirectly access physical address but not directly.

Mapping Virtual Addresses to Physical Addresses

Memory consists of large array addresses. It is the responsibility of the CPU to fetch instruction address from the program counter. These instructions may cause loading or storing to specific memory address.

Address binding is the process of mapping from one address space to another address space. Logical addresses are generated by CPU during execution whereas physical address refers to location in a physical memory unit (the one that is loaded into memory). Note that users deal only with logical address (virtual address). The logical address is translated by the MMU. The output of this process is the appropriate physical address of the data in RAM.

An address binding can be done in three different ways:

Compile Time – If at compile time you know where a process will reside in memory

then an absolute address can be generated - that is a physical address is generated in the program executable during compilation. Loading such an executable into memory is very fast. But if the generated address space is occupied by other process, then the program crashes and it becomes necessary to recompile the program to use a virtual address space.

Load time – If it is not known at the compile time where process will reside then relocatable addresses will be generated. The loader translates the relocatable address to absolute address. The base address of the process in main memory is added to all logical addresses by the loader to generate absolute address. If the base address of the process changes then we need to reload the process again.

Execution time- The instructions are already loaded into memory and are processed by the CPU. Additional memory may be allocated and/or deallocated at this time. This process is used if the process can be moved from one memory to another during execution (dynamic linking-Linking that is done during load or run time). e.g – Compaction.

Memory Management Unit (MMU)

The run time mapping between virtual address and physical address is done by a hardware device known as MMU.

In memory management, Operating System will handle the processes and moves the processes between disk and memory for execution. It keeps the track of available and used memory.

SWAPPING IN OPERATING SYSTEM

Swapping is a memory management scheme in which any process can be temporarily swapped from main memory to secondary memory so that the main memory can be made available for other processes. It is used to improve main memory utilization. In secondary memory, the place where the swapped-out process is stored is called swap space.

The purpose of the swapping in [operating system](#) is to access the data present in the hard disk and bring it to [RAM](#) so that the application programs can use it. The thing to remember is that swapping is used only when data is not present in [RAM](#).

Although the process of swapping affects the performance of the system, it helps to run larger and more than one process. This is the reason why swapping is also referred to as memory compaction.

The concept of swapping is divided into two more concepts: Swap-in and Swap-out.

- o Swap-out is a method of removing a process from RAM and adding it to the hard disk.
- o Swap-in is a method of removing a program from a hard disk and putting it back into the main memory or RAM.

Example: Suppose the user process's size is 2048KB and is a standard hard disk where swapping has a data transfer rate of 1Mbps. Now we will calculate how long it will take to transfer from main memory to secondary memory.

1. User process size is 2048Kb
2. Data transfer rate is 1Mbps = 1024 kbps
3. Time = process size / transfer rate
4. = 2048 / 1024
5. = 2 seconds
6. = 2000 milliseconds
7. Now taking swap-in and swap-out time, the process will take 4000 milliseconds.

Advantages of Swapping

1. It helps the CPU to manage multiple processes within a single main memory.
2. It helps to create and use virtual memory.
3. Swapping allows the CPU to perform multiple tasks simultaneously. Therefore, processes do not have to wait very long before they are executed.
4. It improves the main memory utilization.

Disadvantages of Swapping

1. If the computer system loses power, the user may lose all information related to the program in case of substantial swapping activity.
2. If the swapping algorithm is not good, the composite method can increase the number of Page Fault and decrease the overall processing performance.

Note:

- o In a single tasking operating system, only one process occupies the user program area of memory and stays in memory until the process is complete.
- o In a multitasking operating system, a situation arises when all the active processes cannot coordinate in the main memory, then a process is swap out from the main memory so that other processes can enter it.

Memory Allocation

- Contiguous Memory Allocation
- Non-Contiguous Memory Allocation

In Contiguous Memory Allocation whenever any user process request for the memory then a single section of the contiguous memory block is allocated to that process according to the requirements of the process.

Contiguous memory allocation is achieved just by dividing the memory into the fixed-sized partition.

In Non-contiguous memory allocation, a process is allowed to acquire several memory blocks at different locations in the memory according to its need.

Memory Partitioning

Contiguous memory allocation can be achieved when we divide the memory into the following types of partitions:

- Fixed Partition
- Variable Partition

Fixed partitioning is defined as the system of dividing memory into non-overlapping sizes that are fixed and unmovable. Another name for this is static partitioning. In this case, the system gets divided into multiple fixed-sized partitions. In this type of scheme, every partition may consist of exactly one process. This very process limits the extent at which multiprogramming would occur, since the total number of partitions decides the total number of processes

Variable partitioning is the system of dividing memory into non-overlapping but variable sizes. Dynamic partitioning is another name for this. The scheme allocation in this type of partition is done dynamically. Here, the size of every partition isn't declared initially. Only once we know the process size, will we know the size of the partitions. But in this

case, the size of the process and the partition is equal; thus, it helps in preventing internal fragmentation.

Partition Allocation Methods

The following are four common memory management techniques. There are different Placement Algorithms:

First Fit

Best Fit

Worst Fit

Next Fit

First Fit: In the first fit, the partition is allocated which is the first sufficient block from the top of Main Memory. It scans memory from the beginning and chooses the first available block that is large enough. Thus, it allocates the first hole that is large enough.

Best Fit: Allocate the process to the partition which is the first smallest sufficient partition among the free available partition. It searches the entire list of holes to find the smallest hole whose size is greater than or equal to the size of the process

Worst Fit: Allocate the process to the partition which is the largest sufficient among the freely available partitions available in the main memory. It is opposite to the best-fit algorithm. It searches the entire list of holes to find the largest hole and allocate it to process.

Next Fit: Next fit is similar to the first fit but it will search for the first sufficient partition from the last allocation point.

Pros of Contiguous Memory Allocation

- It supports a user's random access to files.
- The user gets excellent read performance.
- It is fairly simple to implement.

Cons of Contiguous Memory Allocation

- Having a file grow might be somewhat difficult.
- The disk may become fragmented.

Paging in Operating System

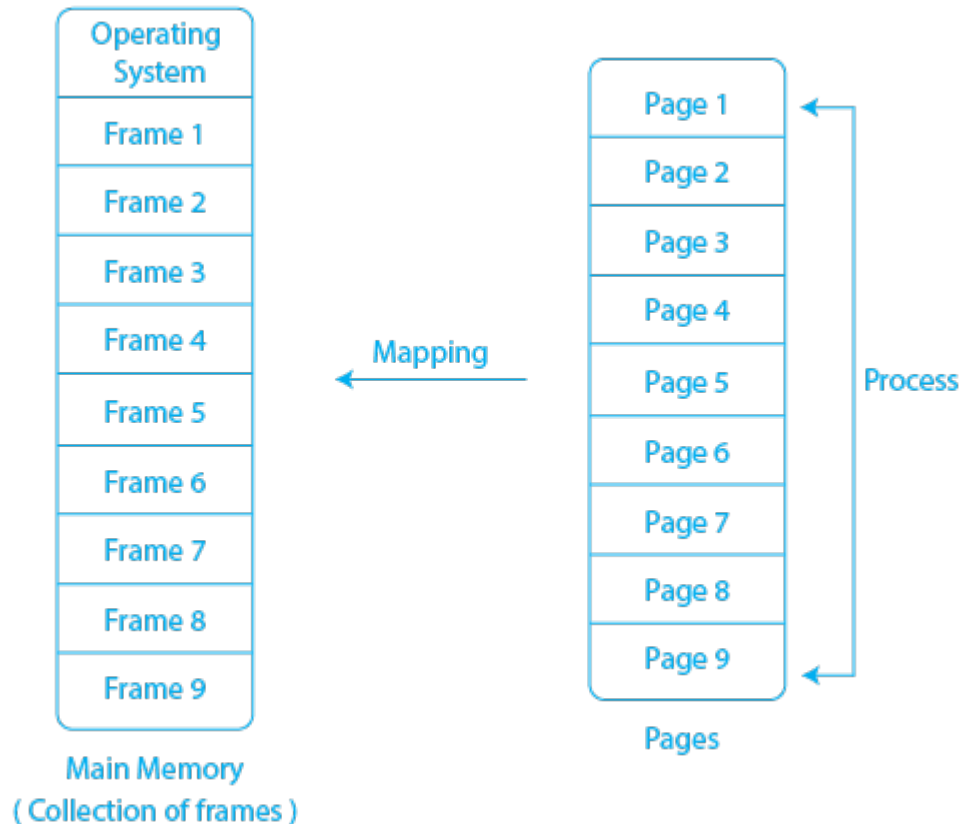
Paging is a memory management scheme that eliminates the need for a [contiguous allocation](#) of physical memory. The process of retrieving processes in the form of pages from the secondary storage into the main memory is known as paging. The basic purpose of paging is to separate each procedure into pages. Additionally, frames will be used to split the main memory. This scheme permits the physical address space of a process to be non – contiguous.

In paging, the physical memory is divided into fixed-size blocks called page frames, which are the same size as the pages used by the process. The process's logical address space is also divided into fixed-size blocks called pages, which are the same size as the page frames. When a process requests memory, the operating system allocates one or more page frames to the process and maps the process's logical pages to the physical page frames.

The mapping between logical pages and physical page frames is maintained by the page table, which is used by the memory management unit to translate logical addresses into physical addresses. The page table maps each logical page number to a physical page frame number.

One process page needs to be saved in each of the available memory frames. These pages may be kept in different parts of memory, but the basic objective is always to locate continuous frames or holes. Process pages are typically only saved in secondary storage until they are required to be in the main memory.

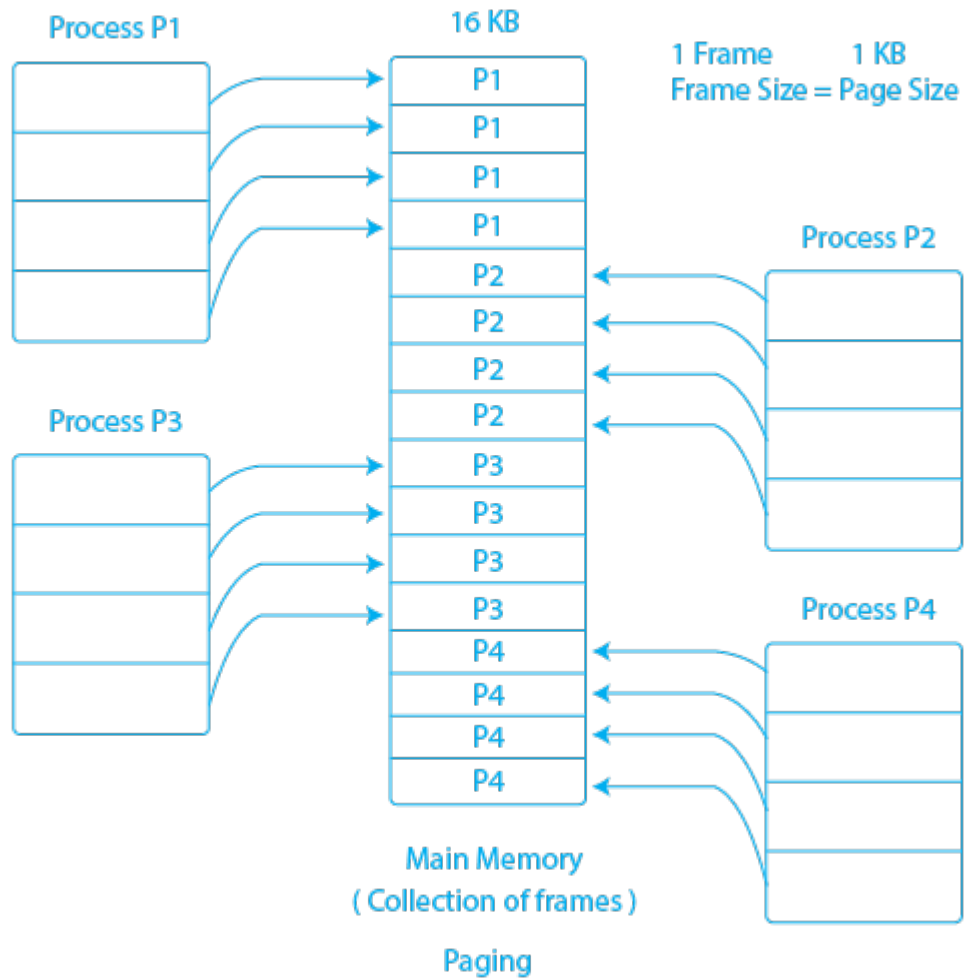
The OS may influence the frame sizes. Every frame needs to have the exact same size. The page size should be similar to the frame size since the pages used in paging are mapped into the frames.



Example of Paging in OS

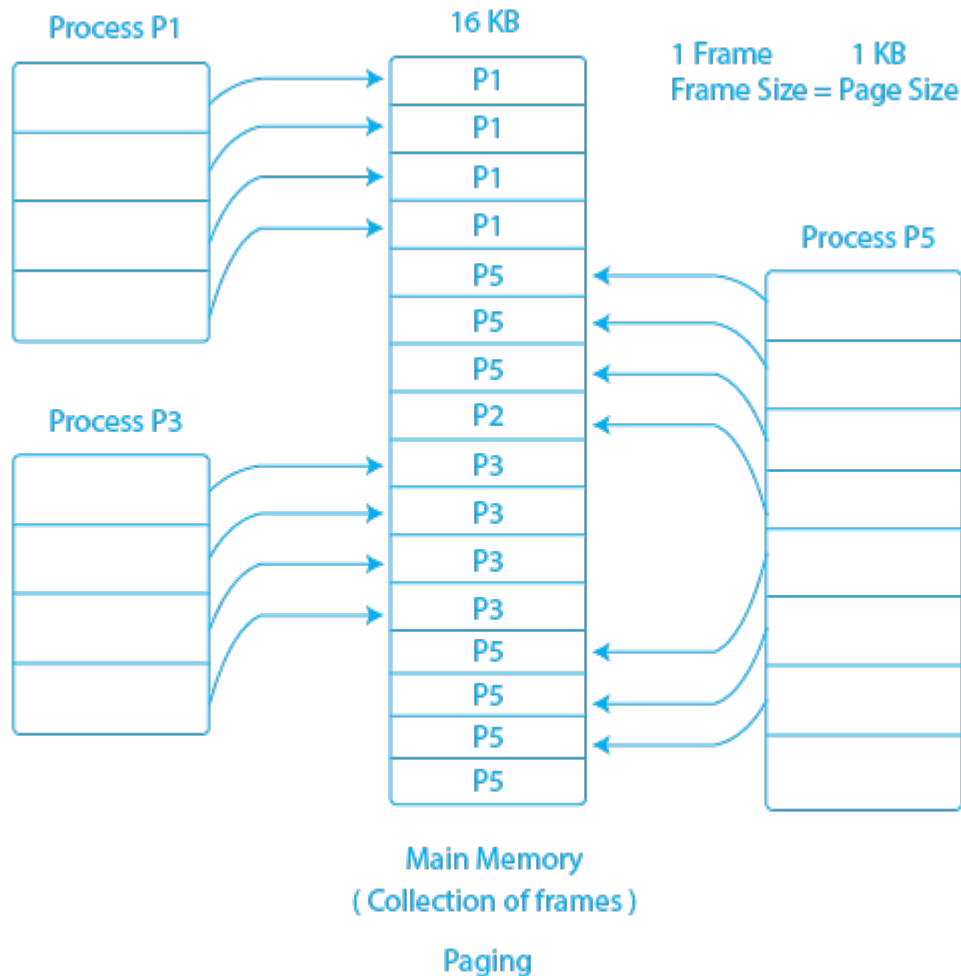
The main memory will be divided into a group of 16 1 KB frames if the main memory is 16 KB and the frame size is 1 KB. The four processes in the system are P1, P2, P3, and P4. They are each 4 KB in size. One page can be saved in a single frame because each procedure is divided into 1 KB pages.

The pages of the processes will be continuously stored because all of the frames are originally empty. Frames, pages, and their mapping are shown in the diagram below.



Think about the scenario where P2 and P4 are eventually sent to the waiting state. Now that eight frames are empty, other pages can be loaded in their place. The process P5, which is 8 KB (8 pages) in size, is located in the ready queue.

We can save the process in multiple places thanks to paging because there are 8 noncontiguous frames that are available in memory. So, rather than loading pages P2 and P4, we can load page P5.



SEGMENTATION IN OPERATING SYSTEM

A process is divided into Segments. The chunks that a program is divided into which are not necessarily all of the exact sizes are called segments. Segmentation gives the user's view of the process which paging does not provide. Here the user's view is mapped to physical memory.

Types of Segmentation in Operating System

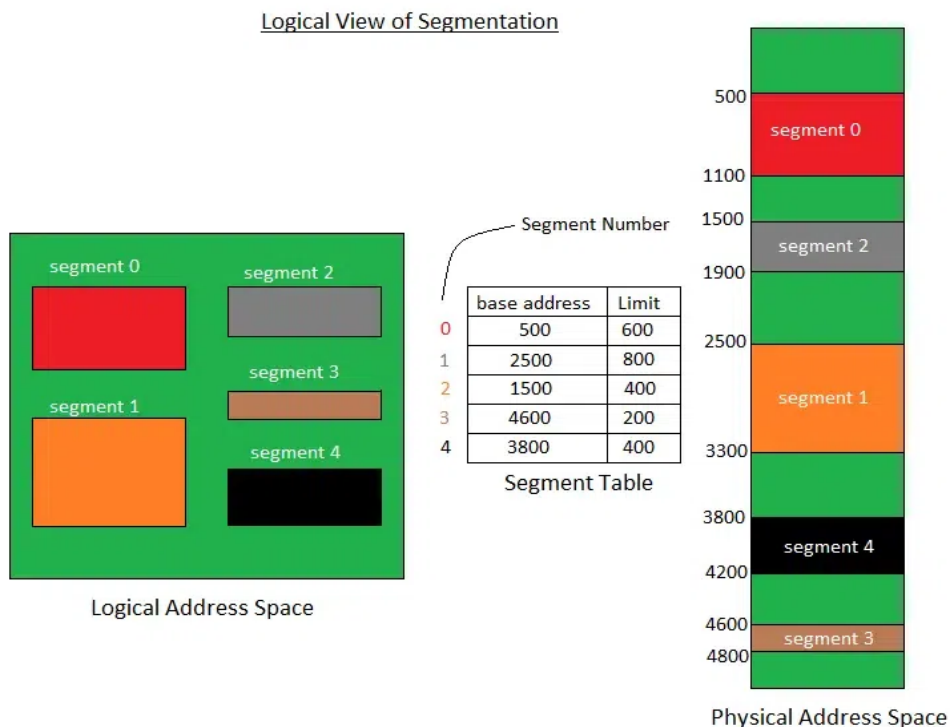
- **Virtual Memory Segmentation:** Each process is divided into a number of segments, but the segmentation is not done all at once. This segmentation may or may not take place at the run time of the program.
- **Simple Segmentation:** Each process is divided into a number of segments, all of which are loaded into memory at run time, though not necessarily contiguously.

There is no simple relationship between logical addresses and physical addresses in segmentation. A table stores the information about all such segments and is called Segment Table.

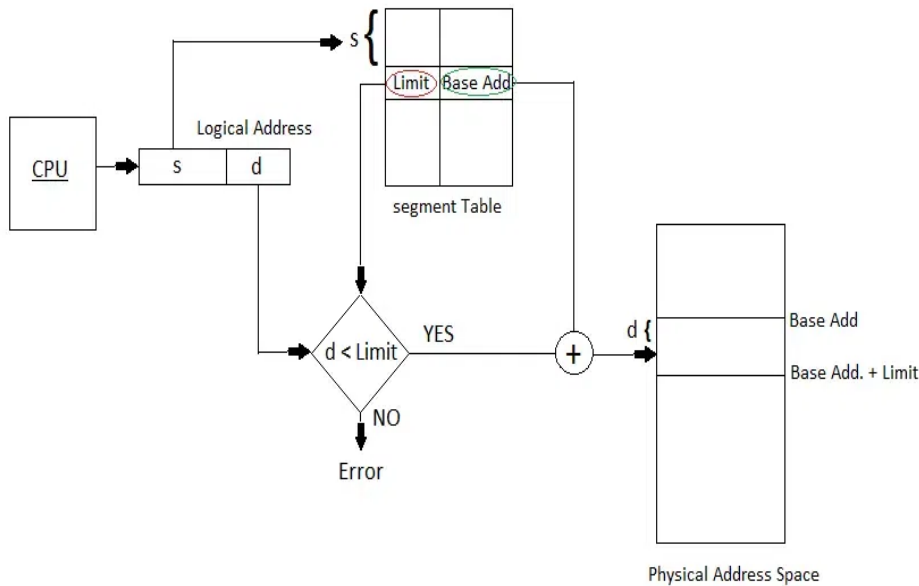
What is Segment Table?

It maps a two-dimensional Logical address into a one-dimensional Physical address. It's each table entry has:

- **Base Address:** It contains the starting physical address where the segments reside in memory.
- **Segment Limit:** Also known as segment offset. It specifies the length of the segment.



Translation of Two-dimensional Logical Address to Dimensional Physical Address.



The address generated by the CPU is divided into:

- **Segment number (s):** Number of bits required to represent the segment.
- **Segment offset (d):** Number of bits required to represent the size of the segment.

Advantages of Segmentation in Operating System

- No Internal fragmentation.
- Segment Table consumes less space in comparison to Page table in paging.
- As a complete module is loaded all at once, segmentation improves CPU utilization.
- The user's perception of physical memory is quite similar to segmentation. Users can
- Divide user programs into modules via segmentation. These modules are nothing more than separate processes' codes.
- The user specifies the segment size, whereas, in paging, the hardware determines the page size.
- Segmentation is a method that can be used to segregate data from security operations.
- **Flexibility:** Segmentation provides a higher degree of flexibility than paging. Segments can be of variable size, and processes can be designed to have multiple segments, allowing for more fine-grained memory allocation.
- **Sharing:** Segmentation allows for sharing of memory segments between

processes. This can be useful for inter-process communication or for sharing code libraries.

- **Protection:** Segmentation provides a level of protection between segments, preventing one process from accessing or modifying another process's memory segment. This can help increase the security and stability of the system.

Disadvantages of Segmentation in Operating System

- As processes are loaded and removed from the memory, the free memory space is broken into little pieces, causing External fragmentation.
- Overhead is associated with keeping a segment table for each activity.
- Due to the need for two memory accesses, one for the segment table and the other for
- Main memory, access time to retrieve the instruction increases.
- **Fragmentation:** As mentioned, segmentation can lead to external fragmentation as memory becomes divided into smaller segments. This can lead to wasted memory and decreased performance.
- **Overhead:** Using a segment table can increase overhead and reduce performance. Each segment table entry requires additional memory, and accessing the table to retrieve memory locations can increase the time needed for memory operations.
- **Complexity:** Segmentation can be more complex to implement and manage than paging. In particular, managing multiple segments per process can be challenging, and the potential for segmentation faults can increase as a result.